

Manipulação de erros

O código Dart pode lançar (throw) ou pegar (catch) exceções ou erros.

```
void main() {  
    throw 'Erro!';  
}
```

Throw

Quando acontecer uma exceção:

```
void main() {  
    throw FormatException('Faltou um argumento!');  
}
```

try

Podemos usar `try...on` para executar algo que pode gerar um erro:

```
void main() {  
  try {  
    adicionarTarefa();  
  } on SemTarefa {  
    criarTarefa();  
  }  
}
```

- aula9_throw1.dart

catch

catch pode receber o erro como argumento.

```
void main() {  
  try {  
    adicionarTarefa();  
  } on SemTarefa catch(e) {  
    print('Ouve um erro: $e');  
    criarTarefa();  
  }  
}
```

- aula9_throw2.dart

finally

finally sempre será executado, com ou sem erro.

```
void main() {  
  try {  
    adicionarTarefa();  
  } on SemTarefa catch(e) {  
    print('Ouve um erro: $e');  
    criarTarefa();  
  } finally {  
    terminarTarefas();  
  }  
}
```

- aula9 throw3.dart

Programação assíncrona

O programa continua enquanto aguarda uma operação terminar.

Operações assíncronas

- Buscar dados da internet
- Ler e escrever arquivos
- Acessar banco de dados

Resultado

O resultado de uma operação assíncrona é:

- `Future`
- `Stream` (se tiver várias partes)

A programação assíncrona usa:

- `Future`
- `async`
- `await`

Exemplo imediato

```
String reservaMesa() {  
    var mesa = buscaMesa();  
    return 'Sua mesa é: $mesa';  
}
```

```
String buscaMesa() {  
    return '7';  
}
```

```
void main() {  
    print(reservaMesa());  
}
```

Prática

Altere o método `buscaMesa()` para retornar na String um número aleatório entre 1 e 10.